

TLM Reference Book

面向 Library Kit 管理者与设计工程师的双视角参考书

工具：TechLib Package Management Kit (仓库 tlm / 命令前缀 tlp_*)
版本对应：V2020_0410a，自述 TLP Management Utility v2020.0410
代码基线：gitee.com/icdop/tlp == github.com/icdop/tlm，commit 8072710
配套文档：《TLM 使用手册》（12 章 + 4 附录，含 32 条“已知限制与坑”实测清单）。本书不重复其全部论证，只给出面向岗位的用法、每个命令的示例，以及一个贯穿芯片设计全流程的管理案例。
手册日期：2026-09-04

本书的可信度声明

书中每一条命令、每一段输出、每一个报错都在下列环境真实执行过，不是从 README 抄写的：

Linux x86_64 · csh 20240808 · GNU Awk 5.2.1 · GNU tar · tree
一套自建的 T28HPC/Op5 演示产品线：6 个基础包 + 3 个 hotfix 包 + 1 个重打包冲突用例

演示用的目录树（本书第四部分案例即沿用它）：

```
/workspace          ← 记作 $ROOT
├── tlm/              ← 工具体本 (TLP_HOME)
├── drop/T28HPC/Op5/  ← Foundry / IP vendor 送来的裸 design kit 目录 (管理者输入)
├── release/          ← 管理者工作区
│   ├── lists/*.csv   ← 发布清单 (11 列 PACKAGE 格式)
│   ├── configs/*.tlp ← 生成的包定义 (发布物)
│   ├── packages/*.tgz ← 生成的包实体 (发布物)
│   ├── dataSheets/   ← catalog (tlp_import 产物)
│   └── templib/      ← tlp_pack 暂存区
├── designer/techLib/ ← 设计工程师的工作库 (安装产物)
├── minimal/          ← 另一工程师的最小工作集
├── mnt_release_tier1/ ← 只读共享盘镜像 (冒号多源演示用)
└── archive/projAlpha_T28HPCOp5_TAPEOUT/ ← Tapout 冻结档案 (自校验、可复现)
```

角色与阅读路径

你的角色	你必须读	你最好读	你可以跳过
Kit 管理者 (Library/CAD Admin, 负责收包、发包、维护 catalog)	Ch1–Ch5、Ch8、Ch10、Ch12	Ch6 (理解设计师会做什么)、Ch9	Ch11
设计工程师 (用 techLib 跑综合/布线/验证)	Ch1、Ch3、Ch6、Ch9、Ch11、Ch12 §12.5–12.7	Ch5 (看懂 catalog)、Ch10 (知道管理者在干什么)	Ch4、Ch7
项目/流程负责人 (管基线 & 审计)	Ch2、Ch10.6、Ch12	Ch3、Ch11.5	其余命令细节

三条贯穿全书的原则（先记住，后面全部例证都围绕它们）：

1 顺序即依赖。TLM 的 DEPN 只有特定写法才生效，真正的依赖管理靠 bundle 清单的书写顺序。

- 2 标记即状态。TLM 没有数据库，一切“装没装”都看目录里的 `.dts` 标记与 `techLib/.tlp_install/` 链接。手工改动 = 改变 TLM 眼中的现实。
 - 3 冻结即真相。一个项目能复现的唯一凭据是 `bundle + configs + packages + md5` 四件套，缺一项就不算冻结。
-

目录

第一部分 · 共同基础

- 第 1 章 角色、职责与数据所有权
- 第 2 章 目录规划与命名规范
- 第 3 章 数据对象参考

第二部分 · 命令参考

- 第 4 章 `tlp_pack` — 打包（管理者）
- 第 5 章 `tlp_import` — 建目录（管理者）
- 第 6 章 `tlp_install` — 安装（管理者与设计者共用）
- 第 7 章 `tlp_check` / `tlp_help` / `make` / `setup.cshrc`
- 第 8 章 扩展命令集（补齐 TLM 缺的能力）
- 第 9 章 选项与环境变量总参考

第三部分 · 角色手册

- 第 10 章 Kit 管理者手册
- 第 11 章 设计工程师手册

第四部分 · 综合案例

- 第 12 章 一颗 28nm SoC 的 Library 资料全生命周期

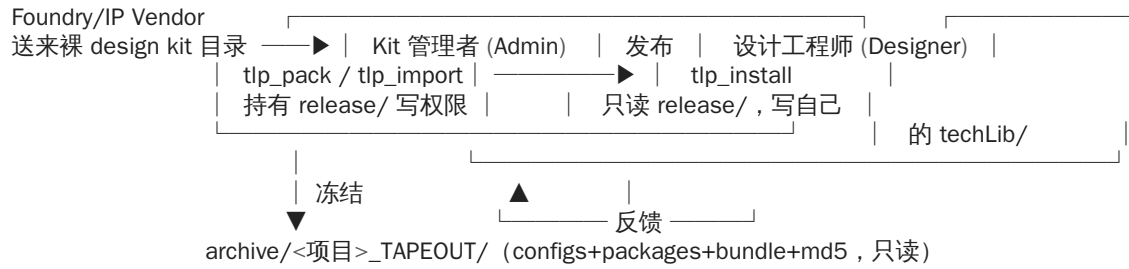
附录

- 附录 A 管理者一页速查卡
 - 附录 B 设计工程师一页速查卡
 - 附录 C 场景 → 命令矩阵
-

第一部分 · 共同基础

第 1 章 角色、职责与数据所有权

1.1 两个角色，一条流水线



1.2 职责边界 (RACI 摘要)

事项	Kit 管理者	设计工程师	项目负责人
接收/解挡 Foundry 交付 (drop/)	R/A	I	—
编写发布清单 (lists/*.csv)	R/A	C (提需求)	—
生成 .tlp + .tgz (tlp_pack)	R/A	—	—
维护 catalog (tlp_import → dataSheets/)	R/A	—	—
决定“哪些包允许被这个项目用”	C	R (自选)	A (冻结时)
安装/升级自己的 techLib/	C	R/A	—
Tapout 冻结四件套	R	C	A
事后 ECO 追加 hotfix	R/A	R (自行安装)	A

R=执行 A=负责 C=需咨询 I=需知会

1.3 数据所有权 (谁可以改，谁绝对不许改)

路径	属主	他人权限	铁律
drop/	管理者	无	原始交付，永不修改内容，只读挂载或 <code>chmod -R a-w</code>
release/lists/、release/configs/、release/packages/	管理者	只读	设计师不得直接改 .tlp；要改请提需求
release/dataSheets/ (catalog)	管理者 (由 import 生成)	只读	任何人手改都会导致“装出来的东西和清单不一致”
archive/*_TAPEOUT/	项目负责人	只读	冻结后不允许任何写入： <code>chmod -R a-w configs packages</code>

路径	属主	他人权限	铁律
<个人>/techLib/	设计工程师	无	只有本人（和 EDA 工具）可写
<项目>/...bundle	项目共管	读	改动 = 换版本，不覆盖旧文件

唯一需要双方共同遵守的东西是命名规范（第 2 章）与 bundle 的顺序语义（第 6 章）。

1.4 双方共用的三条心智模型

- 1 .t1p（管理者写）→ .dts（import 归类）→ <SKU>.dts（install 落到库目录里当“已装凭证”）→ .t1p_install/<SKU>.t1p（符号链接，充当依赖判据）→ <NODE>/<MVER>/t1p_packages（=你的 bundle 来源）。一条链理解清楚，两个角色的操作就都不神秘了。
- 2 TLM 不会替你排序、不会校验 md5、不会卸载。这三件事要么手工做，要么用第 8 章的扩展命令。
- 3 出错先查三处：t1p_install.log、techLib/.t1p_install/、<NODE>/<MVER>/t1p_packages。

第 2 章 目录规划与命名规范

2.1 安装布局（固定，不可配置）

\$TECHLIB_ROOT/\$NODE/\$MVER/\$CATG/\$TYPE/\$SDIR/...
\$TECHLIB_ROOT/\$NODE/\$MVER/.t1p_packages
\$TECHLIB_ROOT/.t1p_install/<SKU>.t1p
\$TECHLIB_ROOT/.t1p_install.summary
\$TECHLIB_DOCS/\$NODE/\$MVER/\$CATG/\$TYPE/\$SDIR/<SKU>.dts
\$TECHLIB_DOCS/.t1p_package_info.csv

← 库内容 + <SKU>.dts 标记
← 该节点已装清单
← 已装凭证（符号链接）
← 安装流水账
← catalog
← catalog 索引

四类目录值请项目启动时一次性定死（例：本书演示产品）：

维度	取值示例	说明
NODE	T28HPC	一个平台一个名字，别混制程细节
MVER	Op5、Op9	版本号里不能有点，用 Op5 表示 0.5（点会破坏某些 EDA 工具的路径解析）
CATG	FDK / FIP / HIP	找厂套件 / 代工厂 IP / 自研 IP—— 决定谁能有权发布
TYPE	PDK CTK ADF STDCELL MEMORY GPIO DDR SERDES	建议锁死一张表，禁止各自发明（AMS 之类旧写法已在样例中被 GPIO 取代）

关键字必须是 CATG。写 GROUP（README 与仓库自带样例里的老写法）会被静默忽略，catalog 立刻长出 T28HPC/Op5//PDK/ 这种空层目录，按类别检索的脚本从此漏包。

2.2 SKU 命名规范（双方签字遵守）

<来源前缀><NODE 简写><TYPE>_<库体>_<版本串>

来源前缀：PT=Foundry 交付 SC=标准单元 MEM=存储 GPIO/IPx=自研 IP

版本串：r<maj>.<min>[HF<n>] 或 e<maj>.<min>.<patch>

PT28HPCPDK_r1.0HF7 SC6T_base_e.2.0 GPIO_e.0.6.1 MEM2PRF_r.0.6.1

PDK_new pdk_r1.0_2023-04 CTK_final r1.0 hf7 (带空格)

PT28HPCPDK_r1.0HF7 与 PT28HPCPDK_r1.0hf7 并存 (大小写混用 = 两个包)

五条硬规定：

- 1 .tlp 文件名 == NAME 字段 == bundle 清单里的那一行 (TLM 到处按文件名推 SKU)。
- 2 SDIR == .tgz 解压出的唯一顶层目录名。不等会“标记在 A、内容在 B”，下一族包直接 conflict root。
- 3 一个 SDIR 只属于一条版本链；链上第一个包 FULL，后续全 PATCH，SDIR 保持一致。
- 4 版本串大小写统一 (建议 hotfix 用 HF 大写)：r1.0hf7 与 r1.0HF7 在仓库样例里真的同时存在，TLM 视为两个不同包。
- 5 发布后 SKU 不可变 (内容变了必须升名)。做到这条，chmod a-w 才有意义。

2.3 共享盘与权限布局模板

```
/tlppkg/          ← 管理者写，全员只读
├── drop/<NODE>/<MVER>/    ← Foundry 原始交付 (只读)
├── release/<NODE>/<MVER>/ ← 当前发布：lists/ configs/ packages/ dataSheets/
├── hotfix_staging/        ← 未转正的 hotfix 包 (可放这里先试装)
└── archive/<PROJ>_<NODE>_<MVER>_<MILESTONE>/ ← 冻结四件套，chmod -R a-w
```

TECHLIB_PKGS 支持冒号分隔的多个源，按顺序查找——于是「稳定层 + 试验层」天然成立 (实测 9/9 包全部从只读源装出)：

```
setenv TECHLIB_PKGS /tlppkg/release/T28HPC/Op5/packages:/tlppkg/hotfix_staging
```

实测 (注意源路径必须绝对，相对路径在 csh 拼接里不展开)：

```
$ TECHLIB_PKGS="$R/mnt_release_tier1:$R/mnt_hotfix_staging" tlp_install \
--packageCfgDir release/configs --targetLibDir srcdemo --bundleList /tmp/full.bundle
: Package file - /tmp/qwenwork/refbook/mnt_release_tier1/PT28HPCKITPDK_r0.6.1.tgz
INFO: Unpacking file '/tmp/qwenwork/refbook/mnt_release_tier1/PT28HPCKITPDK_r0.6.1.tgz' ...
...
[tlp_install]: Total 9/9 kits are installed.      ← 再跑一次变成 9/9 skipped (幂等)
```

只读源当包源是安全的：TLM 只对源做 test -e 与 gunzip -c，从不写入源目录 (演示盘 dr-xr-xr-x 亦从物理上挡住)。

2.4 路径的隐藏红线

- 路径里绝对不能有空格。实测无论命令怎么走，只要目录名含空格，tlp_option.csh 就把参数切碎并抛 setenv: Too many arguments.。用 _/- 命名挂载点。
- 默认值 (techLib、dataSheets、packages) 全是相对路径，脚本没有 cd 保护：换目录没注意，就会在当前目录另起一套 techLib/。团队环境一律显式给绝对路径。
- 挂载点若含符号链接，tlp_pack 的 basename LOCATION 判断 (§4.5) 可能拿到不同值；源目录请用真实路径。

第 3 章 数据对象参考

七种对象的“谁能创建/能否手改/被谁消费”一览——这是两个角色之间最容易打架的地方。

对象	创建者	消费者	人手改？	备注
lists/*.csv	管理者	tlp_pack	正常编辑	11 列，见 §4.2
configs/<SKU>.tlp	tlp_pack 或人写	tlp_import/tlp_install	小改可以，但改完必须重新 import	语法见 §4.4
packages/<SKU>.tgz	tlp_pack	tlp_install	绝对禁止	内部顶层目录名 = SDIR
dataSheets/.../<SKU>.dts	tlp_import	tlp_install (交互)		内容 = 对应 .tlp 的原样拷贝
techLib/.../<SKU>.dts	tlp_install	安装状态判定 + 人查版本	只能整体删/整体补 (§8.5)	删了 = 重装报 conflict
techLib/.tlp_install/<SKU>.tlp	tlp_install	依赖检查		符号链接指向上一条
*.bundle	tlp_pack (末序 = 打包序) / 人 (从 .tlp_packages 抽)	tlp_install -b	但只能追加，不能排序	排序会毁掉依赖顺序，见 §6.7

3.1 .tlp 与 .dts 的真实关系

.dts 就是 .tlp 的一份带路径语义的副本 (tlp_import 只 cp，不改内容)。由此得到两条实用技巧：

```
# ① 在 techLib 里就地查“这个库是哪个包、什么版本、装了什么”
more techLib/T28HPC/Op5/FDK/PDK/pdk28_r061/PT28HPCKITPDK_r0.6.1.dts

# ② 想看 catalog 里某包的定义，不用回溯到 configs/
find release/dataSheets -name 'PT28HPCKITCTK*.dts' -exec grep -l HF1 {} \;
```

3.2 三种清单的区别 (务必分清)

文件	谁生成	顺序含义	典型用途
release/lists/*.csv	管理者手维护	逐行按 FULL/PATCH 打包	打包输入
<NODE>_<MVER>.bundle	tlp_pack (落在 CWD)	打包序 (≈ 依赖序)	发布集合、给设计师范例 bundle
techLib/<NODE>/<MVER>/.tlp_packages	tlp_install 追加	实际安装序	冻结/复现的金标准来源

3.3 一次完整安装留下的状态 (实测)

```
techLib/
├── .tlp_install/          ← 6 个符号链接 = 6 个已装 SKU (权威)
├── .tlp_install.summary  ← "时间 用户 % tlp_install <SKU>\t;<源路径>" 逐行
├── T28HPC/Op5/
│   ├── .tlp_packages     ← 6 行 SKU (= 你的 bundle)
│   └── FDK|FIP|HIP/.../<SDIR>/{内容..., <SKU>.dts}
```

```
$ cat designer/techLib/.t1p_install.summary
20260904_104923 user % t1p_install PT28HPCKITPDK_r0.6.1      ;release/configs/PT28HPCKITPDK_r0.6.1.t1p
20260904_104923 user % t1p_install PT28HPCKITCTK_r0.6.1      ;release/configs/PT28HPCKITCTK_r0.6.1.t1p
...
$ ls designer/techLib/.t1p_install | tr '\n' ' '
GPIO_e.0.6.1.t1p MEM2PRF_r.0.6.1.t1p PT28HPCKITADF_r0.6.1.t1p
PT28HPCKITCTK_r0.6.1.t1p PT28HPCKITPDK_r0.6.1.t1p SC6T_base_e.1.0.t1p
```

这三处就是「我的 techLib 到底是什么」的全部真相，也是任何排错的第一现场。

第二部分 · 命令参考

每个命令条目统一按 用途 → 角色 → 语法 → 选项 → 输入 → 输出 → 实例 → 报错与坑 编排。
所有实例都可复制执行；\$ROOT 见卷首目录树。

第 4 章 t1p_pack — 打包（管理者）

4.1 用途与角色

把「Foundry 送来的裸 design kit 目录」+「一份 11 列 CSV 清单」变成发布物：每个 kit 一个 <SKU>.t1p 与 <SKU>.tgz，并生成 <csv名>.bundle。设计工程师永远不需要碰这个命令。

4.2 语法

```
t1p_pack [--packCfgsDir <出.t1p目录>] [--packDestDir <出.tgz目录>] [--tempDir <暂存目录>]
[-i|--info] [-l <日志>] <list.csv> ...
```

选项	作用	不设时
--packCfgDir / --packCfgsDir	.t1p 输出目录（→ TLP_CFGS_DEST）	packages/（与 tgz 同目录！）
--packDestDir	.tgz 输出目录（→ TLP_PKGS_DEST）	packages/
--tempDir	目录名归一化用的暂存区（→ TECHLIB_TEMP）	tempLib/
-l <f>	日志（必须第一个选项）	t1p_pack.log

强烈建议永远显式给这三个目录。不显式给时 .t1p 与 .tgz 混在 packages/ 里，t1p_import 会把它们当同一批处理，后续维护会痛。

4.3 CSV 清单格式（PACKAGE 1.0）

```
BEGINPACKAGE1.0
NODET28HPC          ← 默认值，可被每行的第 2 列覆盖
MVEROp5
#-----PACKNODEMVERCATGTYPENAMESDIRVERSIONREQULOCATIONCONTENT
FULLT28HPCOp5FDKPKDKPT28HPCKITPDK_r0.6.1pdk28_r061r0.6.1-drop/T28HPC/Op5/FDK/pdk28_r061
FULLT28HPCOp5FDKCTKPT28HPCKITCTK_r0.6.1ctk28_r061r0.6.1PT28HPCKITPDK_r0.6.1drop/T28HPC/Op5/FDK/ctk28_r061
END
```

列位固定 字段名 1 2 3 4 5 6 7 8 9 10 11 :

列	含义	落入 .tlp	说明
1	FULL / PATCH / MULTI	BEGIN TLP <列1>	决定安装期的目录冲突策略 (MULTI 见 §4.7 警告)
2-5	NODE MVER CATG TYPE	NODE/MVER/CATG/TYPE	安装路径四层
6	SKU	NAME + 文件名	必须唯一, 见 §2.2
7	SDIR	SDIR	也决定 tgz 顶层目录
8	版本串	丢弃	不写进 .tlp, 版本只能长在 SKU 名里
9	REQU 基线 SKU	DEPN (见 §4.6 补丁)	-/_= 无依赖; 填自己=自依赖 (会被忽略)
10	LOCATION	决定打包动作	见 §4.5
11	CONTENT	tar 范围	仅当 LOCATION 是目录且 basename≠SDIR 时用, 默认!

注释行用 # 开头 (会被解析器跳过) ; ; 只是“恰好不匹配”, 不要当作可靠注释手段。

4.4 实例 1 : 一个基础版本的完整发布 (run: pack)

```
setenv TLP_HOME $ROOT/tlm
set path = ($TLP_HOME/bin $path)
cd $ROOT
tlp_pack --packCfgsDir release/configs --packDestDir release/packages \
--tempDir release/tempLib release/lists/T28HPC_r061.csv
```

输出 (真实) :

```
INFO: --packCfgDirrelease/configs
INFO: --packDestDirrelease/packages
INFO: --tempDirrelease/tempLib
[tlp_pack]: Processing TechLib Package file 'release/lists/T28HPC_r061.csv' ...
[1]: Packing Kit 'PT28HPCKITPDK_r0.6.1' ...
    Create package file 'PT28HPCKITPDK_r0.6.1.tgz' ..
[2]: Packing Kit 'PT28HPCKITCTK_r0.6.1' ...
    Create package file 'PT28HPCKITCTK_r0.6.1.tgz' ..
...
[tlp_pack]: Total 6/6 tlp packages are created.
```

产物与自查 :

```
$ ls release/configs release/packages      ← 6 个 .tlp + 6 个 .tgz
$ cat release/configs/PT28HPCKITPDK_r0.6.1.tlp
BEGINTLPFULL

NAMEPT28HPCKITPDK_r0.6.1
ORIG_
NODET28HPC
MVEROp5
CATGFDK
```



```
TYPEPDK
SDIRpdk28_r061

FILEPT28HPCKITPDK_r0.6.1.tgz

END
$ cat T28HPC_r061.bundle          ← ★bundle 落在 CWD，不在 lists/ 里
PT28HPCKITPDK_r0.6.1
PT28HPCKITCTK_r0.6.1
...
$ tar tzf release/packages/PT28HPCKITPDK_r0.6.1.tgz | head -1
pdk28_r061/          ← 顶层目录 == SDIR
```

.bundle 写在当前工作目录（源码用 basename 削掉了 CSV 的路径）。要么先 cd 到你想放 bundle 的目录，要么打包完立刻 mv。这是极容易踩的一个坑。

4.5 LOCATION（第 10 列）五种写法的落地行为

写法	pack 动作	典型场景
目录，basename == SDIR	就地 (cd 父; tar -c -O -z SDIR) > 包.tgz	厂里交付目录名已规范
目录，basename != SDIR	先拷进 tempLib/.../<SDIR>/再打包（会重命名顶层目录）	hotfix 碎片目录
指向一个现成 .tgz	直接引用，不重新压缩	外部已有分卷
空 / - / _	只产 .tlp（FILE 指向不存在的名字）	先建档、后补内容
不存在	ERROR: Kit directory '...' does not exist. + 清理暂存	路径拼错

第 2 种写法（basename 不等）在输出里表现为 Copy designkit to kit directory '...'。若两行 CSV 指向同一 LOCATION（例：同一 base 目录发两个变体），后一次会覆盖同名 tgz；tempLib 里的中间目录每次都被 rm -rf 掉，可用 --tempDir 隔离。

4.6 实例 2：hotfix 发布（PATCH 链）

只把改动的几个文件做成一个 patch 包，SDIR 必须与基线一致。CSV 含制表符，请用 bash 生成（csh 的 here-doc 会额外吞入定界符/转义 \t，实测在 csh 下写出的 CSV 会带一个多余 EOF 行且 \t 变成字面量）：

```
cd $ROOT
# 1) 准备差异内容目录（顶层名随便起，SDIR 列会把它归一化成基线名）
mkdir -p drop/T28HPC/Op5/FDK/ctk28_r061_Patch_hf1 && echo "fix hf1" > drop/T28HPC/Op5/FDK/ctk28_r061_Patch_hf1
# 2) 写 hotfix 清单：注意 SDIR 沿用基线 ctk28_r061
cat > release/lists/CTK_hf.csv <<-'EOF'
BEGINPACKAGE1.0
NODET28HPC
MVEROp5
#----PACKNODEMVERCATGTYPENAMESDIRVERSIONREQULOCATIONCONTENT
PATCHT28HPCOp5FDKCTKPT28HPCKITCTK_r0.6.1HF1ctk28_r061r0.6.1HF1PT28HPCKITCTK_r0.6.1drop/T28HPC/Op5/FDK/c
PATCHT28HPCOp5FDKCTKPT28HPCKITCTK_r0.6.1HF2ctk28_r061r0.6.1HF2PT28HPCKITCTK_r0.6.1HF1drop/T28HPC/Op5/FD
END
EOF
```

```

cat > lists/chk.sh <<'EOS'
# 发布清单自查：列数 → 列位 → LOCATION 存在性
awk -F'\t' '
  /^(FULL|PATCH|MULTI|BASE)\t/ {
    if (NF<10 || NF>11) { printf " 第%d行 列数=%d (应 10 或 11) \n", NR, NF; e=1; next }
    if ($4!="FDK" && $4!="FIP" && $4!="HIP")
      { printf " 第%d行 $4=%s 不是 CATG → 列位串了\n", NR, $4; e=1 }
    if ($10=="")      printf " 第%d行 LOCATION 为空 (只出 .tlp 不出包) \n", NR
  }
  END{ printf "%s\n", (e ? " 列检查失败" : " 列数与列位检查通过") }' "$1"
awk -F'\t' ' /^(FULL|PATCH|MULTI|BASE)\t/ && NF>=10 && $10!="" {print $10}' "$1" | sort -u |
while read -r d; do [ -e "$d" ] || echo " LOCATION 不存在: $d"; done
EOS
bash lists/chk.sh release/lists/CTK_hf.csv

```

实测：本书三份清单全部；仓库自带的 run/00_pack/pdk222_r10hf.csv 会被第二关抓出
LOCATION 不存在: pdk222_r061 —— 它少写了 NODE/MVER 两列，\$6 实际是 TOPDIR、\$10
实际是 ORIGIN，
列位整体左移（\$4 恰好还是 FDK，所以光看 CATG 一关发现不了）。
这就是「清单必须严格 11 列」最省事的看门狗。

(tlp_pack 那一行照旧在 csh 里执行：)

```

tlp_pack --packCfgsDir release/configs --packDestDir release/packages --tempDir release/tempLib release/lists/CTK_hf.csv
[1]: Packing Kit 'PT28HPCKITCTK_r0.6.1HF1' ...
[2]: Packing Kit 'PT28HPCKITCTK_r0.6.1HF2' ...
[tlp_pack]: Total 2/2 tlp packages are created.

```

生成出来的 .tlp (打了 §4.7 补丁后的样子)：

```

BEGINLPPATCH
NAMEPT28HPCKITCTK_r0.6.1HF1
...
SDIRctk28_r061          ← 与基线同名，安装时叠加进同一目录
DEPNKIT=PT28HPCKITCTK_r0.6.1
FILEPT28HPCKITCTK_r0.6.1HF1.tgz
END

```

CTK_hf.bundle 内容是 HF1、HF2 (顺序 = 打补丁顺序)。这就是 §6.6 “沿链升级”要用的清单。

4.7 必做的一次性修补 (否则你发布的包没有依赖保护)

tlp_pack 默认产出的是两字段 DEPN <SKU>，而安装端只认三字段/带空白的形式 → 依赖形同虚设。改一行：

```

cd $TLP_HOME
sed -i 's|"DEPN\t"kit_depend|"DEPN\tKIT=\t"kit_depend|" csh/tlp_pack.awk
sed -n '141p' csh/tlp_pack.awk      # print "DEPN\tKIT=\t"kit_depend >> kit_tlp

```

验证 (重新打包后)：

```

$ grep -c 'DEPNKIT=' release/configs/*.tlp | tr '\n' ' '
GPIO_e.0.6.1.tlp:1 MEM2PRF_r.0.6.1.tlp:1 PT28HPCKITADF_r0.6.1.tlp:1 ...

```

配合它，安装端才有这样的行为 (实测)：

```

: Required kit 'PT28HPCKITPDK_r0.6.1' has not been installed yet.
ERROR: Can not install 'GPIO_e.0.6.1' (dependency fail)

```

KIT= 后面必须还有空白。写成 DEPN

KIT=<SKU> (无空白) 会让安装端读到空字段，从而该包永远装不上（报 Required kit '' has not been installed yet.）。《使用手册》§5.3 有五种写法的完整实测矩阵。

4.8 报错与坑 (pack)

现象	原因	对策
跑了但像什么都没做	CSV 列数不是 11（如仓库自带 pdk222_r10hf.csv 只 8 列）→ 列位串到 NODE=FDK、TYPE=r1.0.1	严格 11 列；awk -F'\t' 'NF>11' lists/*.csv 自查
依赖形同虚设	默认产两字段 DEPN	§4.7 补丁
.bundle 找不到	落在 CWD	先 cd 或打包后 mv
想发布多分卷的 MULTI 行只出一条 FILE	MULTI/PATCH 分支是死代码（判的是未赋值的 pack_type），实测 9 个 tgz 目录只写成 1 条 FILE，且行为退化为“重新打包该目录”	分卷请直接手写多行 FILE 的 .t1p（《使用手册》§5.4 样例 C）
MODE TEST 一开全表都变演练	它是 CSV 级开关（BEGINFILE 复位）	演练单独一个 CSV
CSV 里；注释有时被当数据	；不匹配任何关键字正则才恰好安全	统一用 #
ERROR: Kit directory '...' does not exist.	LOCATION 拼错/未挂载	用绝对路径或先 test -d
打包极慢 / 磁盘满	中间会在 --tempDir 复制整份 design kit（第 2 类 LOCATION）	把 tempDir 指到大盘；或让交付目录名直接等于 SDIR

第 5 章 t1p_import — 建目录（管理者）

5.1 用途与角色

把 .t1p 登记进 catalog (dataSheets/)：按 NODE/MVER/CATG/TYPE/SDIR 归类，同时充当发布门禁——载荷 tgz 或 REQU 基线 .t1p 任一缺失，这个包不会进入 catalog（没有 .dts，设计师在交互菜单里就看不到它）。由管理者执行，设计师只需要会读它的产物。

5.2 语法

```
t1p_import [--packageCfgDir <放.t1p>] [--packageSrcDir <放.tgz>] [--dataSheetDir <catalog>]
[-i|--info] [-l <日志>] [ <文件.t1p> | <目录> | <SKU名> ... ]
```

输入形态四种都支持（实测：目录里有 9 个 .t1p 时，“无参”与“传该目录”都导入 9 个；“单文件”与“裸 SKU 名”各导 1 个）：

你给什么	它做什么
（什么都不给）	ls -1 \$TECHLIB_CFGS/*.t1p 全量导入

你给什么	它做什么
一个目录	展开为该目录下 *.t1p
一个 .t1p 路径	只导它（增量发布常用）
一个裸 SKU 名	到 TECHLIB_CFGS 通配 <SKU>.t1p

目录里没有任何 .t1p 时，会抛 `csH` 原生的 `ls: No match.` 然后静默收尾（退出码仍是 0）——很容易误以为“导入成功”。

5.3 实例 1：全量导入一次发布

```
cd $ROOT/release
t1p_import --packageCfgDir configs --packageSrcDir packages --dataSheetDir dataSheets

[1]: Reading 'configs/GPIO_e.0.6.1.t1p' ...
: Package type - FULL
: Package file - packages/GPIO_e.0.6.1.tgz
: Creating 'T28HPC/Op5/HIP/GPIO/gpio_e06/GPIO_e.0.6.1.dts' (GPIO_e.0.6.1)
...
[t1p_import]: Total 6/6 t1p data sheets are created.

$ find dataSheets -name '*.dts' | sort
T28HPC/Op5/FDK/ADF/adf28_r061/PT28HPCKITADF_r0.6.1.dts
T28HPC/Op5/FDK/CTK/ctk28_r061/PT28HPCKITCTK_r0.6.1.dts
T28HPC/Op5/FDK/PDK/pdk28_r061/PT28HPCKITPDK_r0.6.1.dts
T28HPC/Op5/FIP/MEMORY/mem2p_r06/MEM2PRF_r.0.6.1.dts
T28HPC/Op5/FIP/STDCELL/sc6t_e10/SC6T_base_e.1.0.dts
T28HPC/Op5/HIP/GPIO/gpio_e06/GPIO_e.0.6.1.dts
```

索引文件同时生成（制表符分隔，实测）：

```
$ head -2 dataSheets/.t1p_package_info.csv
T28HPC Op5 HIP GPIOGPIO_e.0.6.1 gpio_e06 GPIO_e.0.6.1 _
T28HPC Op5 FIP MEMORYMEM2PRF_r.0.6.1 mem2p_r06 MEM2PRF_r.0.6.1 _
```

5.4 实例 2：增量导入 + 幂等

```
t1p_import --packageCfgDir configs --packageSrcDir packages --dataSheetDir dataSheets
```

（第二次跑同一份 configs）

真实输出（ECO 那一轮：新增 1 个、其余全跳过）：

```
WARNING: Skip import T28HPC/Op5/HIP/GPIO/gpio_e06/GPIO_e.0.6.1.dts (already exist)
: Creating 'T28HPC/Op5/FDK/CTK/ctk28_r061/PT28HPCKITCTK_r0.6.1HF3.dts' (PT28HPCKITCTK_r0.6.1HF3)
-----
[t1p_import]: Total 1/9 t1p data sheets are created.
[t1p_import]: Total 8/9 t1p files are skipped. (exist)
-----
```

要点：

- 已存在的 .dts 不会被更新。若你改了 configs/<SKU>.t1p 的内容再重导，只会看到 `WARNING: Kit '...' in the DK_RELN has been modified`（只告警不改写）。
- 正确做法是升 SKU；如果确实是同 SKU 修错（发错了包），必须先手删该 .dts 再 import：

```
find release/dataSheets -name 'PT28HPCKITCTK_r0.6.1.dts' -delete
t1p_import --packageCfgDir release/configs --packageSrcDir release/packages --dataSheetDir release/dataSheets
```

并且要立刻通知已安装该包的人：他们库里的旧 `.dts` 标记不会自动变。

5.5 实例 3：门禁拒收（管理者最需要的行为）

故意用一个 `FILE` 指向不存在 `tgz` 的定义：

```
tlp_import --packageCfgDir errs2 --dataSheetDir ds_errs
      : Install base - NO_SUCH_BASE_KIT can not be found in package source.
      : Pacakge file - NO_SUCH_FILE.tgz can not be found in package source.
ERROR: Kit 'BADKIT_missing' has 2 missing pacakge files
-----
[tlp_import]: Total 0/1 tlp data sheets are created.
[tlp_import]: Total 1/1 tlp files have missing package files. (error)
```

→ 只要出现 `(error)`，这个包就进不了 `catalog`。把它当 CI 关卡（退出码恒 0，必须 `grep` 日志/输出，见 §8.2）。

5.6 CI 用法：只导新增的、并拒绝任何 `error`

```
tlp_import --log ci_import.log --packageCfgDir release/configs \
--packageSrcDir release/packages --dataSheetDir release/dataSheets \
release/configs/PT28HPCKITCTK_r0.6.1HF3.tlp
grep -q '(error)' ci_import.log && exit 1 # ★注意 --log 必须是第一个选项
```

5.7 报错与坑（import）

现象	原因	对策
Is: No match. + 什么都没导入	TECHLIB_CFGS 指错（默认等于 TECHLIB_PKGS）	永远显式给 <code>--packageCfgDir</code>
导进去了但没成功，退出码还是 0	恒 <code>exit 0</code>	<code>grep (error)</code> (§5.6)
改了 <code>.tlp</code> 但 <code>catalog</code> 没变	已存在的 <code>.dts</code> 只告警不更新	升 SKU，或 §5.4 手删后重导
<code>catalog</code> 出现 <code>T28HPC/Op5//PDK/</code> 空层	<code>.tlp</code> 用了 <code>GROUP</code> 而非 <code>CATG</code>	修 <code>.tlp</code> ；发布前跑 §8.2 <code>gate.sh</code>
<code>.tlp_package_info.csv</code> 字段数忽 4 忽 5	该行同时用空格与 Tab 分隔	用 <code>awk -F'[\t]+'</code> ，或干脆用 <code>find -name '*.dts'</code> 代替解析
相对路径下重复导入生成两套 <code>catalog</code>	所有默认值都是相对路径	一律绝对路径
有 <code>DEPN</code> 却没做检查	<code>import</code> 根本不解析 <code>DEPN</code> （只解析 <code>REQU/FILE</code> ）	想校验依赖完整性，用 <code>gate.sh/install</code> 期

第 6 章 `tlp_install` — 安装（管理者与设计者共用）

6.1 用途与模式选择

唯一“把包变成可用库”的命令。管理者用它建参考实现/验证发布，设计工程师用它建与升级自己的工作库。

给了包名 / --bundleList → 批处理模式 (可无人值守、可进 CI)
什么都没给 → 交互菜单模式 (从 catalog 里挑, 需要终端)

6.2 语法

```
tlp_install [-l <日志>] [--targetLibDir <techLib>] [--packageCfgDir <放.tlp>]  
            [--packageSrcDir <放.tgz>] [--dataSheetDir <catalog>]  
            [-b|--bundleFile|--bundleList <清单>] [-s|--selectByCategory <类别>]  
            [ <SKU> | <path>/x.tlp | <path>/x.dts ... ]
```

语法红线 (全部实测)

- 1 选项必须全部在位置参数之前: `tlp_install X --targetLibDir tl` ⇒ `ERROR: Can not find TLP config '--targetLibDir'.`
- 2 `--log` 必须是第一个选项 (被 `tlp_header.csh` 消化, 其后选项会被当包名)。
- 3 路径不能含空格; `TECHLIB_PKGS` 的冒号多源要用绝对路径。
- 4 `-s/--selectByCategory` 不做任何筛选, 只是把值当作交互提示里的默认显示 (实测: 给了它照样弹 `INPUT: Category = (T28HPC/Op5/FDK)?` 等输入)。README 描述的“预选并列出”未实现。
- 5 `-v/--verbose` 完全无效 (`awk` 里 `for (option in arr)` 比的是下标; `-v` 还会被 `head` 吃掉, `CMDSD:` 行里看不见它)。

6.3 实例 1 (设计工程师) : 按发布 bundle 一键建工作库

```
cd $ROOT  
tlp_install --packageCfgDir release/configs --packageSrcDir release/packages \  
            --targetLibDir designer/techLib --bundleList T28HPC_r061.bundle  
  
[1]: Checking tlp package - PT28HPCKITPDK_r0.6.1 ...  
...  
INFO: Unpacking file 'release/packages/PT28HPCKITPDK_r0.6.1.tgz' ...  
[tlp_install]: Total 6/6 kits are installed.
```

结果 (真实目录) :

```
designer/techLib/  
├── .tlp_install/{6 个 <SKU>.tlp 符号链接}  
├── .tlp_install.summary  
└── T28HPC/Op5/  
    ├── .tlp_packages ← 6 行, 顺序 = 安装顺序  
    ├── FDK/{ADF/adf28_r061, CTK/ctk28_r061, PDK/pdk28_r061}  
    ├── FIP/{MEMORY/mem2p_r06, STDCELL/sc6t_e10}  
    └── HIP/GPIO/gpio_e06
```

6.4 实例 2 : 包名寻址的三种写法 + 找不到时什么样

```
tlp_install --packageCfgDir release/configs --packageSrcDir release/packages \  
            --targetLibDir designer/techLib GPIO_e.0.6.1 # ① 裸 SKU : 去 CFGS 找 <SKU>.tlp  
tlp_install ... --targetLibDir designer/techLib release/configs/MEM2PRF_r.0.6.1.tlp # ② 直接给 .tlp 路径  
tlp_install ... --targetLibDir designer/techLib release/dataSheets/T28HPC/Op5/FDK/CTK/ctk28_r061/PT28HPCKITCTK_r0.6.1
```

寻址顺序: 原样当文件 → `$TECHLIB_CFGS/<名>` → `$TECHLIB_CFGS/<名>.tlp` → 失败。找不到即整批中止:

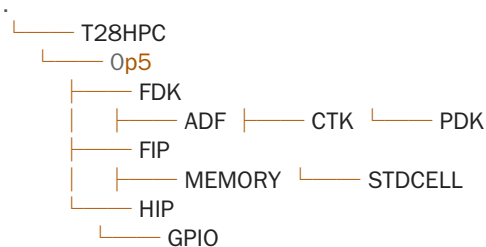
```
[1]: Checking tlp package - NO_SUCH_KIT ...  
ERROR: Can not find TLP config 'NO_SUCH_KIT' in 'release/configs'.
```

⇒ 把不确定的名字先用 `tlp_import <SKU名>` 试一遍, 或用 §11.4 的查询指令确认。

6.5 实例 3：交互菜单（现场最常用）

```
tlp_install --packageCfgDir release/configs --packageSrcDir release/packages \
--dataSheetDir release/dataSheets --targetLibDir inter
```

INFO: Please specify Kit Category :



INPUT: Category = () ? T28HPC/Op5/FDK

INFO: Please select the following packages to be installed :

[release/dataSheets/T28HPC/Op5/FDK]:

0) Go back to previous selection menu..

# SKU	:TYPE	:TOPDIR
1) PT28HPCKITADF_r0.6.1	:ADF	:adf28_r061
2) PT28HPCKITCTK_r0.6.1	:CTK	:ctk28_r061
3) PT28HPCKITCTK_r0.6.1HF1	:CTK	:ctk28_r061
4) PT28HPCKITCTK_r0.6.1HF2	:CTK	:ctk28_r061
5) PT28HPCKITCTK_r0.6.1HF3	:CTK	:ctk28_r061
6) PT28HPCKITPDK_r0.6.1	:PDK	:pdk28_r061

h) hide installed packages.. t) print directory tree.. q) quit..

INPUT: Select ? 6

选完 6（PDK 基线）后，列表里它变成带星号，表示已安装：

* 6) PT28HPCKITPDK_r0.6.1 :PDK :pdk28_r061

h) hide installed packages..

INPUT: Select ? h ← 按 h 后该行从列表消失，只剩未装的

菜单按键一览：

键	作用	注意
3 2 1	一行输入多个序号，按你写的顺序逐个装	顺序错 → 依赖失败
0	回上一层改类别	
h / a	隐藏 / 显示已装项 (*)	
t	tree -n \$TECHLIB_DOCS/<类别> less	需要 tty
q	退出（直接 exit 0，日志里不会有 END 行）	
非法/越界	ERROR: Invalid selection : X / selection over the range : (1~N)	

选错顺序的真实后果（先点 HF1，基线未装）：

```
[1]: Reading 'release/dataSheets/.../PT28HPCKITCTK_r0.6.1HF1.dts' ...
: Package type - PATCH
: Required kit 'PT28HPCKITCTK_r0.6.1' has not been installed yet.
```



```
WARNING: Base directory 'ctk28_r061' is missing for patch package.
ERROR: Can not install 'PT28HPCKITCTK_r0.6.1HF1' (dependency fail)
```

交互模式每选一项就立刻装一项，中途 q 会留下半成品
techLib（已装的仍在清单里）。稳妥习惯：交互里只做“试装/演示”，正式环境一律走 bundle。
无人值守要重放交互（例如回归）：把回答写进文件喂给 stdin —— printf
'T28HPC/Op5/FDK\n6\nq\n' | tlp_install ...。

6.6 实例 4：hotfix 升级（PATCH 叠加进同一目录）

```
tlp_install --packageCfgDir release/configs --packageSrcDir release/packages \
--targetLibDir designer/techLib --bundleList CTK_hf.bundle

: Directory 'designer/techLib/T28HPC/Op5/FDK/CTK/ctk28_r061' already exist.
INFO: Unpacking file 'release/packages/PT28HPCKITCTK_r0.6.1HF1.tgz' ...
: Directory 'designer/techLib/T28HPC/Op5/FDK/CTK/ctk28_r061' already exist.
INFO: Unpacking file 'release/packages/PT28HPCKITCTK_r0.6.1HF2.tgz' ...
[tlp_install]: Total 2/2 kits are installed.
```

升级带来的文件级差异（find ... -type f 前后对比）：

```
> ctk28_r061/FIX.hf1
> ctk28_r061/FIX.hf2
> ctk28_r061/PT28HPCKITCTK_r0.6.1HF1.dts
> ctk28_r061/PT28HPCKITCTK_r0.6.1HF2.dts
```

跳链安装被拦（清单里 HF2 依赖 HF1）：

```
: Required kit 'PT28HPCKITCTK_r0.6.1HF1' has not been installed yet.
ERROR: Can not install 'PT28HPCKITCTK_r0.6.1HF2' (dependency fail)
```

再执行同一 bundle 是安全的（幂等）：

```
WARNING: Skip install 'PT28HPCKITCTK_r0.6.1HF1' (already installed)
WARNING: Skip install 'PT28HPCKITCTK_r0.6.1HF2' (already installed)
[tlp_install]: Total 2/2 kits are skipped. (already exist)
```

6.7 实例 5：安装顺序就是依赖管理（★最重要的一节）

把冻结清单按字母排序再装——许多人的“顺手优化”——结果：

```
$ sort -o techLib.bundle techLib.bundle
$ tlp_install ... --bundleList techLib.bundle
[tlp_install]: Total 2/8 kits are installed.
[tlp_install]: Total 6/8 kits require check fail. (error) ← 基线被排到最后，全线崩
```

保持 .tlp_packages 的原始（安装）顺序则正确：

```
$ tlp_install ... --bundleList archive/.../techLib.bundle
[tlp_install]: Total 8/8 kits are installed.
```

规范（写进项目 SOP）：

- 1 baseline bundle = cat techLib/<NODE>/<MVER>/.../techLib.bundle （要清重只允许 awk '!seen[\$0]++' 保序去重）；
- 2 手工写的 bundle 一律基线在前、派生在后；
- 3 不放心顺序就用 §8.4 的 topo.sh 生成，或干脆分两趟装同一 bundle（第一趟装能装的，第二趟补齐）——幂等性保证收敛。

6.8 实例 6：最小工作集（不是所有人都要全套）

```
tlp_install --packageCfgDir release/configs --packageSrcDir release/packages \
--targetLibDir minimal PT28HPCKITPDK_r0.6.1 SC6T_base_e.1.0

T28HPC/Op5/FDK/PDK/pdk28_r061
T28HPC/Op5/FIP/STDCELL/sc6t_e10
```

只装两个包即满足数字前端需求；ADF/GPIO 不装，techLib 就小一大截（真实 PDK 场景下这是几十 GB 差别）。

6.9 实例 7：FULL 撞上已存在的 SDIR（重打包/改名的常见事故）

```
[1]: Reading 'release/configs/CTK_REPACK_r0.6.2.tlp' ...
ERROR: Kit directory 'ctk28_r061' already exist before installing full kit package.
[tlp_install]: Total 1/1 kits have conflict root. (error)
```

三条处置：① 新包本应是 PATCH（改 .tlp 的 BEGIN 行）；② 新包换 SDIR（并保证 tgz 顶层同名）；③ 目标本就是错的旧库 → §8.5 卸载后重灌。

6.10 “已装标记”被手删后的行为（排错必备）

删掉 techLib/.../pdk28_r061/PT28HPCKITPDK_r0.6.1.dts 后再装同一 FULL 包：

```
ERROR: Kit directory 'pdk28_r061' already exist before installing full kit package.
[tlp_install]: Total 1/1 kits have conflict root. (error)
```

补回标记即恢复幂等：

```
cp release/configs/PT28HPCKITPDK_r0.6.1.tlp \
designer/techLib/T28HPC/Op5/FDK/PDK/pdk28_r061/PT28HPCKITPDK_r0.6.1.dts
$ tlp_install ... PT28HPCKITPDK_r0.6.1
WARNING: Skip install 'PT28HPCKITPDK_r0.6.1' (already installed)
```

原因见 §1.4 心智模型 2：TLM 只认标记，不认目录内容。

6.11 安装内部到底做了什么（11 步，用于解释一切现象）

```
1 读 .tlp/.dts → 2 找 tgz(冒号多源) → 3 查 <SKU>.dts 标记(FULL/PATCH 决定 跳过|冲突|放行)
→ 4 查 DEPN(仅 §4.7 的有效写法) → 5 mkdir 类别/SDIR → 6 cp 定义进库目录当标记
→ 7 ln -s 进 .tlp_install/ → 8 追加 .tlp_install.summary → 9 递归装 REQU 基线
→ 10 gunzip|tar 解到 .../CATG/TYPE (所以 tgz 顶层必须是 SDIR) → 11 追加 SKU 到 <NODE>/<MVER>/ .tlp_packages
```

由此可直接推出四条“设计上的限制”：没有回滚（第 10 步逐步写）、没有并发锁、--dry-run 不存在（想预演就装到临时 --targetLibDir 再废弃）、卸载得手工（§8.5）。

6.12 唯一会自动装基线的机制：REQU（★被低估的实用技巧）

TLM 全程没有“依赖求解”，但有一个例外：.tlp 里写 REQU 时，tlp_install 会自己再调一次 tlp_install <基线>.tlp 把基线先装上（实测）：

```
: Install base - reqcfg/PT28HPCKITPDK_r0.6.1.tlp
INFO: Installing base kit 'reqcfg/PT28HPCKITPDK_r0.6.1.tlp' ...
INFO: Unpacking file 'release/packages/PT28HPCKITPDK_r0.6.1.tgz' ... ← 基线自动补齐
[tlp_install]: Total 1/1 kits are installed.
INFO: Unpacking file 'release/packages/PT28HPCKITADF_r0.6.1.tgz' ... ← 然后才装本体
```

但 `REQU` 有两个读者、字段位不同（`import` 读 `$2`、`install` 读 `$3`），名字必须写两遍：

写法	<code>import</code> 校验基线在不在（ <code>\$2</code> ）	<code>install</code> 自动装基线（ <code>\$3</code> ）	<code>topo.sh</code>
<code>REQU <基线></code> <code><基线></code> （两列同名）			取末列
<code>REQU <占位> <基线></code>	报 <code>Install base - <占位></code> <code>can not be</code> <code>found</code> ，整包被门禁拒收		
<code>REQU <基线></code> （只一列）		<code>\$3</code> 空 → <code>Install base -</code> <code>can not be found +</code> <code>(missing</code> <code>pacakge)</code> ，装不上	

⇒ 团队规范：基线关系一律 `REQU <基线SKU> <基线SKU>`；同时保留 `DEPN`（§4.7 产物）给 `topo.sh` 与“人读”。

四条必须知道的限制：

- 1 递归靠 `system("tlp_install ...")`，要求 `tlp_install` 在 `PATH` 上（已 `make bin` 且 `bin` 在 `PATH`），否则基线装不上且报错不明显。
- 2 子进程自己打一份 `SUMMARY`，所以会看见两行 `Total 1/1 kits are installed.`（不是装了两遍）；日志与 `.tlp_packages` 也各多一条，属正常。
- 3 只补一层：基线自己的基线得由基线的 `REQU` 继续往上写，否则链断在半路。
- 4 `REQU` 只看“`<基线>.tlp` 在不在 `CFGS`”，不校验版本——热修链上要写准基线 `SKU`。

收益：有了它，设计师一条命令就能带出基线：

```
tlp_install --packageCfgDir $P/configs --packageSrcDir $P/packages \  
--targetLibDir ~/techLib PT28HPCKITADF_r0.6.1 # PDK 基线被自动补装
```

对比 §6.7（无 `REQU` 时同样命令只会得到 `(dependency fail)`）。建议写进“设计师自助装单个库”的操作说明里。

第 7 章 `tlp_check` / `tlp_help` / `make` / `setup.cshrc`

7.1 `tlp_check`：当它不存在

```
tlp_check --packageCfgDir release/configs --targetLibDir ck PT28HPCKITPDK_r0.6.1
```

实测：0 字节输出、不建目录、不写日志、退出码 0。原因有两层：

- 1 逻辑未实现（源码只做 `banner` + 选项解析 + 写日志）；
- 2 首行 `shebang` 写坏成 `#!/bin/csh -f set verbose=1`，经 `bin/tlp_check` 调用时脚本根本没执行。显式 `csh -f csh/tlp_check.csh ...` 才会走到流程，且它把日志硬编码写到 `tlp_install.log`（会把真实安装日志轮转掉）。

→ 校验请用 §7.4 ; README 里 tlp_check 的三种用法均不可用。

```
$ csh -f $TLP_HOME/csh/tlp_check.csh --targetLibDir ck2 PT28HPCKITPDK_r0.6.1
# TechLib Package (TLP) Management Utility v2020.0410
TIME: @20260904_105309 BEGIN tlp_check.csh
CMDS: tlp_check.csh --targetLibDir ck2 PT28HPCKITPDK_r0.6.1
INFO: --targetLibDirck2
TIME: @20260904_105309 END tlp_check.csh
$ ls ck2 → No such file or directory ← 什么也没检查、什么也没建
$ wc -c tlp_install.log → 1346 → 399 ← 日志被它截断 (轮转成 .1)
```

7.2 tlp_help

```
tlp_help          # 打印用法列表
tlp_help command  # 依次执行 tlp_import/tlp_install/tlp_check --help
tlp_help env | readme | format | example | run | testcase | project | update
```

实测注意三件事：

- tlp_help command 只输出 2 段 Usage: (tlp_check 静默，见 §7.1)。
- tlp_help env 在没 setenv 的情况下直接崩：TECHLIB_ROOT: Undefined variable. (它 echo 未定义变量)。用 make env 更安全。
- readme/format/example/start 指向 \$TLP_HOME/docs/... (实际目录是 doc/，而且没有 PDF，还外部依赖 xpdf) → 基本不可用；run/testcase 会往当前目录 cp -fr run 然后进 run/01_case 跑 make help (会污染你的 CWD)；update 里是 svn update/ci (上古遗留，别执行)。

7.3 make 与 setup.cshrc

```
cd $TLP_HOME && make install    # = make bin : 重建 bin/ 下 5 个符号链接
source $TLP_HOME/setup.cshrc    # 设 TLP_HOME + 3 个 TECHLIB_* 并打印 tlp_help
```

setup.cshrc 的两个问题：TECHLIB_PKGS 指向 \$TLP_HOME/packages (仓库里不存在该目录，照抄会 “No match”)；TECHLIB_DOCS/ROOT 是相对路径。建议只用它设 TLP_HOME 与 PATH，其余自己定：

```
setenv TLP_HOME /tools/tlm
set path = ($TLP_HOME/bin $path)
setenv TECHLIB_CFGS /tlppkg/release/T28HPC/Op5/configs
setenv TECHLIB_PKGS /tlppkg/release/T28HPC/Op5/packages
setenv TECHLIB_DOCS /tlppkg/release/T28HPC/Op5/dataSheets
setenv TECHLIB_ROOT /proj/$USER/techLib
```

7.4 替代 tlp_check 的三件工具 (本书附录/第 8 章提供脚本)

需求	工具	一句话
装完之后库还完整吗	tlp_verify.sh	遍历 .tlp_install 反查标记/目录/载荷
.tlp 写得对不对	gate.sh	名字一致性、CATG、DEPN 形式、tgz 顶层、md5
这批包的装序对吗	topo.sh	按有效依赖拓扑排序

第 8 章 扩展命令集 (补齐 TLM 缺的能力)

TLM 本体没有：md5 校验、依赖求解、卸载、安装后自检、批量导入。第 8 章给出已实测可用的补齐件（与《使用手册》附录 B 同源，这里按岗位重排）。

8.1 tlp_verify.sh — 安装后自检（管理者、设计工程师都要）

```
bash tlp_verify.sh <techLibDir> <tgz源目录>

$ bash tlp_verify.sh designer/techLib release/packages
OK  GPIO_e.0.6.1
OK  MEM2PRF_r.0.6.1
...
---- designer/techLib/.tlp_install 共 9 条记录
RESULT: 全部通过
```

它能抓出人工破坏（实测注入两种故障）：

```
BAD  GPIO_e.0.6.1: .dts 丢失(库目录被手删)
BAD  PT28HPCKITCTK_r0.6.1: 缺载荷 PT28HPCKITCTK_r0.6.1.tgz
RESULT: 校验失败      ← 退出码 1
```

脚本见 §8.6 清单（关键：结果先收进变量再判定——管道里的标志位传不出子 shell）。

8.2 gate.sh — .tlp/.tgz 入仓门禁（管理者专用）

对 release/configs/*.tlp 逐条检查并给出可执行判定（实测对本书演示集运行，正确抓出 hotfix 链的 SDIR/DEPN 形式问题且无误报）：

检查项	报错样例
NAME 与文件名一致	X: NAME 与文件名不一致
存在 CATG	X: 缺 CATG（不要写 GROUP！）
DEPN 是否为有效形式	X: DEPN 不是推荐形式（请用 DEPN KIT <类别> TOPDIR <topdir>）
每条 FILE 的 tgz 存在	X: 缺载荷 a.tgz
tgz 顶层目录 == SDIR	X: tgz 顶层 pdk28_r061 ≠ SDIR
有 MD5S 则实际比对	X: a.tgz md5 不符（…≠…）

8.3 md5 的正规用法（.tlp 里的 MD5S 只是装饰）

安装时唯一有意义的动作：

```
cd /tlppkg/release/T28HPC/Op5/packages && md5sum -c ../packages.md5 | grep -v ': OK'
```

冻结档案里 packages.md5 / configs.md5 都以文件名相对生成，因此进目录再 -c（在目录外跑会全线 FAILED open or read，实测踩过）：

```
$ cd archive/projAlpha_T28HPCOp5_TAPEOUT/packages && md5sum -c ../packages.md5 | tail -1
SC6T_base_e.1.0.tgz: OK
$ cd ../configs && md5sum -c ../configs.md5 | grep -c ': OK'
8
```

8.4 topo.sh — 依赖拓扑排序

```
$ bash topo.sh release/configs $(cat archive/.../techLib.bundle)
PT28HPCKITPDK_r0.6.1      ← PDK 基线自动置顶
GPIO_e.0.6.1
...
```

只识别有效依赖（DEPN KIT= <SKU> 与 REQU）；DEPN KIT <类别> TOPDIR <topdir> 是目录式，不参与排序，此时靠“基线在 bundle 前部”保证。检测循环依赖与缺定义均返回码 1。

8.5 卸载与回退（TLM 没有 uninstall）

```
set SKU = GPIO_e.0.6.1
grep -l "^DEPN.*$SKU" release/configs/*.t1p # ① 谁依赖它（必须锚定 DEPN 行，
# 否则 grep $SKU 会命中该包自己的 NAME 行）
cp -a designer/techLib designer/techLib.bak.$SKU # ② 备份！这是别人的工作现场
rm -rf designer/techLib/T28HPC/Op5/HIP # ③ 删库目录
rm -f designer/techLib/.t1p_install/$SKU.t1p # ④ 删已装凭证（否则依赖检查仍认为它在）
set L = designer/techLib/T28HPC/Op5/.t1p_packages # ⑤ 从已装清单里摘除
grep -v "^$SKU\$" $L > $L.new && mv $L.new $L
bash t1p_verify.sh designer/techLib release/packages # ⑥ 复验
```

实测三步全做 → RESULT: 全部通过（grep -l "^DEPN.*GPIO..." 返回空，说明无人依赖，可安全卸）；
漏做第 ④ 步（只删目录不删凭证）→ .t1p_install 里残留悬空链接：

```
BAD GPIO_e.0.6.1: .dts 丢失(库目录被手删)
RESULT: 校验失败
```

补做 ④ 后立刻恢复通过，且重装该包正常（Total 1/1 kits are installed.）。

⇒ “目录 + .t1p_install/<SKU>.t1p + .t1p_packages 行”三处必须一起删，少一处就是不一致状态。
反例（谁依赖 PDK 基线，卸不得）：GPIO_e.0.6.1、MEM2PRF_r.0.6.1、PT28HPCKITADF_r0.6.1、PT28HPCKITCTK_r0.6.1、SC6T_base_e.1.0。

8.6 脚本清单获取

上述四个脚本的完整源码在《TLM 使用手册》附录 B（已逐个实测）；本书第四部分把它们嵌入到真实流程里演示。

第 9 章 选项与环境变量总参考

9.1 选项 ↔ 变量 ↔ 缺省（一次看全）

短	长	影响	缺省	真实生效？
-l	--log <f>	日志名	t1p_<cmd>.log（自动轮转 .1/.2...）	必须是第一个选项
-v	--verbose	—	—	不生效（§6.2）
-i	--info	echo 四个 TECHLIB_*	—	TECHLIB_CFGS 打印的是 PKGS 的值（显示 bug）
-c	--packageCfgDir	TECHLIB_CFGS	=\$TECHLIB_PKGS	

短	长	影响	缺省	真实生效？
-p	--packageSrcDir	TECHLIB_PKGS (支持 a:b:c)	packages	
-r	--dataSheetDir	TECHLIB_DOCS	dataSheets	
-t	--targetLibDir	TECHLIB_ROOT	techLib	
-b	--bundleFile/--bundleList	批处理清单	空	
-s	--selectByCategory	仅菜单默认显示	空	不筛选
—	--packCfgsDir/--packDestDir/--tempDir	pack 三目录	—	
—	TECHLIB_OPTION (环境变量)	期望传 --verbose/--test	—	下标比较 bug，全失效

9.2 退出码 (写脚本必读)

情形	exit
正常完成	0
有 (error)：缺载荷 / conflict root / 依赖失败 / import 拒绝入库	仍然 0
包名在 TECHLIB_CFGS 找不到、--bundleFile 不存在、--dataSheetDir 目录缺失	1
--help	-1 (即 255)
结论	CI 里必须 grep 输出/日志判定，见 §5.6 与 §12.3

9.3 日志

命令	默认日志
tlp_import	tlp_import.log
tlp_install	tlp_install.log
tlp_pack	tlp_pack.log
tlp_check	tlp_install.log (会污染安装日志)

日志含 ANSI 色码，查看用 `less -R tlp_install.log`，判定用 `sed -e 's/\x1b\[[0-9;]*m//g'`。每次运行会把旧日志轮转成 .1 .2 … (无限累积，记得定时清)。

第三部分 · 角色手册

第 10 章 Kit 管理者手册

你的 KPI：设计师任何时候都能装出一套已知正确的 techLib，且任何时候都能解释某个项目当时用的是哪些包。

10.1 你的日常循环

收 (drop/) → 审 (gate) → 包 (t1p_pack) → 登 (t1p_import) → 验 (装到 /tmp 参考库 + verify)
→ 告 (变更通告) → 冻 (项目里程碑时 archive 四件套) → 计 (磁盘/审计清理)

10.2 SOP：一次常规发布（新 collateral 或新版本）

```
cd /tlppkg && setenv REL release/T28HPC/Op5
```

```
# ① 收：Foundry 交付只读归档
```

```
cp -r /incoming/T28HPC_HSPICE_r1.01 $REL/./drop/T28HPC/Op5/FDK/ctk28_r071  
chmod -R a-w $REL/./drop/T28HPC/Op5/FDK/ctk28_r071
```

```
# ② 写清单（严格 11 列；SKU 按 §2.2；SDIR 与交付目录同名）
```

```
vi $REL/lists/T28HPC_r071.csv
```

```
# ③ 打包 + 门禁 + 导入（三步必须零 error）
```

```
cd $ROOT
```

```
t1p_pack --packCfgsDir $REL/configs --packDestDir $REL/packages \  
--tempDir $REL/tempLib $REL/lists/T28HPC_r071.csv
```

```
mv T28HPC_r071.bundle $REL/bundles/
```

```
bash gate.sh ; echo "gate rc=$status" # csh 用 $status，不是 $?
```

```
t1p_import --log $REL/import_r071.log --packageCfgDir $REL/configs \  
--packageSrcDir $REL/packages --dataSheetDir $REL/dataSheets
```

```
grep -q '(error)' $REL/import_r071.log && echo "发布被门禁拒：载荷/基线缺失" && exit 1
```

```
# ④ 冒烟：装一个一次性参考库（用完就扔，绝不动别人的 techLib）
```

```
t1p_install --packageCfgDir $REL/configs --packageSrcDir $REL/packages \  
--targetLibDir /tmp/reflib_r071 --bundleList $REL/bundles/T28HPC_r071.bundle
```

```
bash t1p_verify.sh /tmp/reflib_r071 $REL/packages || exit 1
```

```
diff -r -x '.t1p_install*' -x '.t1p_packages' /tmp/reflib_r071 $REL/golden/techLib || echo "与上一里程碑有差异，写进通告"
```

```
# ⑤ 发布不可变 + 通告
```

```
chmod -R a-w $REL/packages $REL/configs
```

```
echo "见 §10.7 通告模板"
```

关键点：③ 的 `grep '(error)'` 是唯一可靠的成功判据（TLM 永远返回 0）；④ 的临时目录冒烟是管理者唯一安全的“预演”手段（TLM 没有 `--dry-run`）。

10.3 SOP：hotfix（Foundry 只给了 delta 目录）

- 1 确认它改的是哪个 SDIR —— hotfix 的 SDIR 必须等于基线的 SDIR，否则装出来两个目录。
- 2 BEGIN PACKAGE 清单里第一个（若基线在盘上已存在）用 PATCH；整库重发才用 FULL。
- 3 REQU 填上一个 HF 的 SKU（链式），而不是永远填基线：链式让“跳装”被拦下。
- 4 打完包 → import → 冒烟时必须从项目当时的 bundle 往后装（复用项目 bundle + 新 HF），而不是凭空只装 HF。

冒烟的正确姿势（别只装 HF，要拿项目 bundle 往后接）：

```
set ECO = /tmp/smoke_ECO1.bundle
```

```
cp archive/$PROJ/techLib.bundle $ECO
```

```
echo PT28HPCKITCTK_r0.6.1HF3 >> $ECO # ★追加，不排序
```

```
t1p_install --packageCfgDir $REL/configs --packageSrcDir $REL/packages \  
--targetLibDir /tmp/reflib_r071 --bundleList $REL/bundles/T28HPC_r071.bundle
```



```
--targetLibDir /tmp/reflib_${PROJ} --bundleList $ECO
bash tlp_verify.sh /tmp/reflib_${PROJ} $REL/packages
```

(csh 里没有 heredoc 命令替换, `--bundleList /dev/stdin <<EOF` 那种写法是 bash 习惯, 别用。)

10.4 SOP : 新节点 / 新分组 (避免历史包袱)

新增 MVER=Op9 或 CATG=HIP 时, 不要顺手写进既有 CSV 里。给新组合建独立清单与独立 catalog 快照:

```
lists/          T28HPC_r061.csv T28HPC_r071.csv T28HPC_Op9_r010.csv
configs/        ← 全量都在这里 (历史不删)
bundles/        ← 每个里程碑一个文件
dataSheets/     ← 增量 import 即可 (catalog 只增不改)
```

catalog 会越滚越大, 这是特性不是问题 (老项目仍要引用老包)。控制手段: 每个里程碑 `cp dataSheets/.tlp_package_info.csv` 快照进 `meta/`。

10.5 权限与不可变策略

```
# tier-1 发布盘: 管理者组可写, 其他人只读
chmod 750 /tlppkg/release/T28HPC/Op5 && chgrp libadmin /tlppkg/release/T28HPC/Op5
chmod -R a-w /tlppkg/release/T28HPC/Op5/packages /tlppkg/release/T28HPC/Op5/configs
# 冻结档案: 物理只读 (实测连 rm 都会被拒: Permission denied)
chmod -R a-w archive/PROJ_ALPHA_TAPEOUT
```

同一文件名的覆盖发布绝对禁止 (.tgz 内容变了而 SKU 没变 = 全公司的 techLib 与清单不再可比)。要改就升 SKU。

10.6 审计报表 (管理者 4 条日常查询)

```
# ① catalog 里到底有哪些包 (含版本、SDIR)
awk -F'[ \t]+' 'NF>4{printf "%-8s %-8s %s/%s %s\n", $1, $2, $3, $4, $5}' \
  /tlppkg/release/T28HPC/Op5/dataSheets/.tlp_package_info.csv

# ② 某个 SKU 有没有被登记 (设计师报“菜单里找不到”时先查这个)
find /tlppkg/release/T28HPC/Op5/dataSheets -name 'PT28HPCKITCTK_r0.6.1.dts'

# ③ 哪些包引用了某包 (谁会被你影响)
grep -l '^DEPN.*PT28HPCKITPDK_r0.6.1' /tlppkg/release/T28HPC/Op5/configs/*.tlp

# ④ 各项目用的包组合 (盘点)
grep -h " /proj/*/meta/techLib.bundle | sort | uniq -c | sort -rn | head
```

③ 实测输出 (意味着你若要退役/改动 PDK 基线, 得先通知这 5 个包的 owners) :

```
GPIO_e.0.6.1.tlp MEM2PRF_r.0.6.1.tlp PT28HPCKITADF_r0.6.1.tlp
PT28HPCKITCTK_r0.6.1.tlp SC6T_base_e.1.0.tlp
```

10.7 变更通告模板

[T28HPC/Op5 发布 2026-09-04]
新增: PT28HPCKITCTK_r0.6.1HF3 (依赖 HF2)
内容: <路径>/FIX.hf3 —— 修正 CTS mask 层规则
影响: 使用 CTK ctk28_r061 的所有项目
动作: 把 PT28HPCKITCTK_r0.6.1HF3 追加到你的 bundle 末尾并执行 §11.3 的命令
不可回退项: 已用 HF2 跑过 signoff 的 tapeout 请沿用冻结档案, 勿追新
位置: config=.../configs/PT28HPCKITCTK_r0.6.1HF3.tlp tgz md5=8a1c...

10.8 包退役 (EOL)

- 1 `grep -l` 确认无人依赖且无活跃项目引用 (§10.6 ③④) ；
- 2 在 catalog 中标“deprecated”——但绝不删 `.dts` (删了会让别人重装报 conflict、历史解释不了) ；改在 `configs/<SKU>.tlp` 的 `END` 之后追加人类可读告示 (`END` 之后工具不解析) ：

END
!! DEPRECATED 2026-09 : 请改用 PT28HPCKITCTK_r0.7.1 (r0.6.1 系列的 HF 链已停止维护)
- 3 `packages/*.tgz` 永久保留 (磁盘策略走冷层，不删除) 。

10.9 管理者排错速查

设计师报障	你先看	多数原因
“菜单里没有这个包”	<code>find dataSheets -name '<SKU>.dts'</code>	import 时被门禁拒 (缺 tgz) / 没跑 import
“装不上，说找不到 config”	他的 <code>--packageCfgDir</code>	指到 <code>packages/</code> 去了 (默认值 = <code>TECHLIB_PKGS</code>)
“报 dependency fail”	你 <code>.tlp</code> 的 <code>DEPN</code> 写法 + 他 bundle 顺序	基线没排在前面 (§6.7)
“报 conflict root”	该 <code>SDIR</code> 被谁占	hotfix 写成 <code>FULL</code> ；或新包复用了旧 <code>SDIR</code>
“装完内容像旧的”	两个包的 <code>SDIR</code> 是否相同而 <code>TYPE</code> 不同	tgz 顶层目录名 ≠ <code>SDIR</code> (§4.5)
“ <code>.tlp_packages</code> 有重复行”	多次装同一 <code>SKU</code> 的追加	生成 bundle 时 <code>awk '!seen[\$0]++'</code> 保序去重

第 11 章 设计工程师手册

你只需记住三件事：从 bundle 装、别手改 techLib、出问题先跑 `tlp_verify.sh`。

11.1 我的第一步：建工作库

```
setenv TLP_HOME /tools/tlm
set path = ($TLP_HOME/bin $path)
setenv PROJ Alpha

tlp_install --packageCfgDir /tlppkg/release/T28HPC/Op5/configs \
--packageSrcDir /tlppkg/release/T28HPC/Op5/packages \
--targetLibDir /proj/$PROJ/$USER/techLib \
--bundleList /proj/$PROJ/meta/techLib.bundle
bash tlp_verify.sh /proj/$PROJ/$USER/techLib /tlppkg/release/T28HPC/Op5/packages
```

看到 `RESULT: 全部通过` 再开始干活。
不要在交互菜单里建项目正式库 (§11.6) 。

11.2 我该装哪些？（按岗位的最小集）

岗位	需要的 TYPE	示例 bundle 片段
RTL/验证（仅仿真模型）	CTK（+PDK 基线）	PT28HPCKITPDK_r0.6.1 → PT28HPCKITCTK_r0.6.1
综合/DFT	PDK+STDCELL+MEMORY	再加 SC6T_base_e.1.0、MEM2PRF_r.0.6.1
P&R / signoff	PDK+ADF+STDCELL+MEMORY+GPI O	全套
模拟/定制	PDK+ADF(+CTK)	不含 FIP/HIP 也可以

顺序恒为：PDK 基线 → .dts/模型类 → IP 类 → hotfix 追加。装最小集能省几十 GB 与大量 NFS 压力。

11.3 升级/追 hotfix（只加不改）

```
cd /proj/$PROJ/$USER
echo PT28HPCKITCTK_r0.6.1HF3 >> techLib/T28HPC/Op5/.t1p_packages.new
cat techLib/T28HPC/Op5/.t1p_packages > techLib.bundle.new
cp techLib/T28HPC/Op5/.t1p_packages /proj/meta/techLib.bundle.pre_HF3 # 留回退点
echo PT28HPCKITCTK_r0.6.1HF3 >> /proj/meta/techLib.bundle # ★追加，别排序
t1p_install --packageCfgDir ... --packageSrcDir ... \
  --targetLibDir techLib --bundleList /proj/meta/techLib.bundle
bash t1p_verify.sh techLib .../packages
```

因为 t1p_install 幂等，重跑整份 bundle 也安全（已装的变 Skip install ... (already installed)）：

```
[t1p_install]: Total 2/2 kits are skipped. (already exist)
```

升级后请做“差异确认”，别只信通告：

```
find techLib -newermt '-1 day' -type f ! -name '*.dts' | sort # 昨天起新增/改动的文件
```

11.4 “我的库里到底装了什么？”（三条自查指令）

```
ls techLib/.t1p_install/ | sed 's/\.t1p$/ /' # 权威“已装”集合
cat techLib/T28HPC/Op5/.t1p_packages # 含安装顺序
tail -5 techLib/.t1p_install.summary # 最近谁装了什么（含时间/用户）
```

查某个库的版本/来源定义（.dts 就在库里，TLM 帮你留了自解释能力）：

```
more techLib/T28HPC/Op5/FDK/CTK/ctk28_r061/PT28HPCKITCTK_r0.6.1HF2.dts
```

看 catalog 里可装而本机未装的包：

```
comm -13 <(ls techLib/.t1p_install | sed 's/\.t1p$/ /' | sort) \
  <(find /tlppkg/release/T28HPC/Op5/dataSheets -name '*.dts' -printf '%f\n' \
    | sed 's/\.dts$/ /' | sort -u)
```

11.5 多项目 / 多节点并存的正确姿势

```
/proj/Alpha/zhang/techLib ← 每人一份，用同一个 bundle
/proj/Beta/li/techLib ← 允许不同 bundle（版本隔离天然靠 <NODE>/<MVER> 路径 + bundle）
/proj/Beta/li/techLib_Op9 ← 试装新 MVER：另起 targetLibDir，别混装进 Op5 那份
```

- 不要把两个 MVER 的 bundle 混进同一 targetLibDir（虽然物理上不打架，但“项目用了什么”就说不清了）。
- 对比两个方案时 --bundleList 指不同清单，装到不同 --targetLibDir 即可。
- 磁盘紧张时同一份 techLib 用 冒号多源 + 只读共享盘装（§2.3）。

11.6 交互菜单：只用于探索

在交互菜单中一次安装是立即执行且顺序敏感的（§6.5 演示）。适合“我看看 FDK 下有哪些版本可以选”，不适合建正式库。若一定要用，按这个点击顺序：PDK 基线 → 其他 PDK/CTK → ADF → FIP → HIP → 最后 HF 链。

11.7 EDA 工具怎么指到 techLib

TLM 只负责把库铺成 .../\$CATG/\$TYPE/\$SDIR。工具端的环境变量应统一指向 techLib 根目录，从而在换版本时不用改流程：

```
setenv PDK_PATH /proj/$PROJ/$USER/techLib/T28HPC/Op5/FDK/PDK/pdk28_r061
setenv TLF_PATH /proj/$PROJ/$USER/techLib/T28HPC/Op5/FDK/CTK/ctk28_r061
setenv NETLIST_LIB /proj/$PROJ/$USER/techLib/T28HPC/Op5/FIP/STDCELL/sc6t_e10
```

⇒ 这也解释了 §2.2 里“SDIR 永不改名”的必要性：改了 SDIR = 全公司 EDA 启动脚本作废。

11.8 设计工程师排错速查

屏幕上的话	真实原因	你该做的
Can not find TLP config 'X' in 'configs'	名字错 / 该包没进 catalog / 选项写在了包名后面	用 §11.4 的 comm 查可装集合；挪选项到前面
Required kit '...' has not been installed yet. + (dependency fail)	顺序问题 或 管理者没打 §4.7 补丁而你在跳链装	从 bundle 头开始重装；或找管理者
already exist before installing full kit package. + conflict root	你想装的 FULL 撞了已存在的 SDIR	该装的其实是 PATCH；先查 §11.4 已装集合
Base directory '...' is missing for patch package.	基线没装就上 hotfix	先装基线
Pacakge file - x.tgz can not be found in package source.	包盘没挂 / TECHLIB_PKGS 少了一层源	找管理者；或补冒号多源
kits require check fail 汇总后没有 END 时间戳	你在交互里按了 q	无影响
装完发现少文件	有人手删过库目录	bash tlp_verify.sh ... 定位，再按 §10.2④ 让管理者补发
整份库要重来	—	mv techLib techLib.bad && tlp_install --bundleList ... （别 rm -rf 没备份的）

11.9 设计师 FAQ

Q：为什么我不该 `cp -r` 别人的 techLib？

A：拷过去的 techLib 带着别人的绝对路径符号链接与 `.dts` 标记，看起来能用，但 `.tlp_install/*` 若指向不存在的路径，后续任何“重装/依赖检查/审计”全部不一致，而且没人知道差异。要复用请复用 bundle。

Q：能不能只装一个包不装基线？

A：取决于该包的 DEPN 写法。有效写法（§4.7 之后产物）会直接拒；两字段 DEPN <SKU> 的包会被放行——但那是发布缺陷，别把它当许可（工具不会告诉你缺了什么）。

Q：`.tlp` 我能自己改吗？

A：可以（放本地 `--packageCfgDir` 里自建一套），但不要改共享盘。改完记得 SKU 也要改名，否则别人装到的和你装的不一样。

Q：磁盘紧张能删 `.dts` 或 `.tlp_install` 吗？

A：不能。它们是状态本体（§1.4 模型 2），删了 TLM 就“看不见”已装事实，重装立刻 conflict。

Q：怎么知道我这次仿真用的哪版工艺库？

A：跑 log 采集时顺手 `cat $TECHLIB_ROOT/$NODE/$MVER/.tlp_packages > run.meta/techLib.bundle + md5sum techLib/**/ *.dts`。项目要求冻结时这就是你的凭据。

第四部分 · 综合案例

第 12 章 一颗 28nm SoC 的 Library 资料全生命周期

主角：projAlpha，T28HPC Op5，28nm SoC（数字为主 + 一小块模拟），团队 14 人（1 名 Kit 管理者、9 名设计工程师、项目/流程各 1）。

时间跨度：Kickoff → Tapeout → 回片 ECO → 归档（约 11 个月）。

目标：把前面所有命令放进一个真实流程里走一遍；每个阶段说明

谁做、做什么命令、产出什么、怎么验收、存哪里。

本章的命令与输出与卷首演示树完全对应（书中所有 `Total x/x`、报错、目录都是真实执行结果）。

12.0 全流程一览

阶段	时点	管理者动作	工程师动作	关键产物	验收
P0 立项	W0	定 NODE/MVER /CATG/TYPE 字典、盘权限	—	《库命名与目录规范》	评审签字
P1 首批收货	W2	gate → tlp_pack → tlp_import → 参考库	—	6 个 <code>.tlp+.tgz</code> 、catalog、T28HP C_r061.bundle	import 零 error + verify 通过
P2 环境铺开	W3	发通告、给 bundle	各自 <code>tlp_install -bundleList</code> 、EDA 挂载	9 份个人 techLib	人人 RESULT: 全部通过
P3 迭代/hotfix	W8–W24	PATCH 链发布 (HF1/2/3)	bundle 追加 + 重跑幂等	HF 包 + <code>.dts</code> 标记	差异确认 (find -newermt)

阶段	时点	管理者动作	工程师动作	关键产物	验收
P4 里程碑冻结	W26 (Tapout)	生成四件套、ch mod a-w、ARCH IVE.sha256	提供各自 bundle	archive/projAlph a_T28HPCOp5_ TAPEOUT/	包 8/8、定义 8/8 md5 一致
P5 复现验证	W26	让新人只凭档案 复装	独立复装 + diff -r	repro/、newhire /	逐文件一致
P6 回片 ECO	W38	冻结后新增 HF ，不改冻结档案	在 ECO 分支 bundle 上追加	ECO bundle	verify + 影响面分析
P7 年度审计	W48	盘点目录、清日 志、EOL 标记	确认在用版本	审计表	§10.6 四条查询

12.1 P0 立项：先把“字典”定死（1 天，价值最高）

管理者产出并被评审接受的三张表（内容即 §2.1/§2.2）：

字典 A：NODE=T28HPC MVER=Op5(首版) / Op9(model 大版)
字典 B：CATG=FDK|FIP|HIP；TYPE=PDK|CTK|ADF|STDCELL|MEMORY|GPIO
字典 C：SKU 规范 + “SDIR 永不改名” + “一个 SDIR 唯一归属一条版本链”

并落一次性环境标准（写进 /etc/profile.d/tlm.csh）：

```
setenv TLP_HOME /tools/tlm
set path = ($TLP_HOME/bin $path)
alias tlpverify 'bash /tools/tlm/contrib/tlp_verify.sh $TECHLIB_ROOT $TECHLIB_PKGS'
```

工具侧的一次性修补（§4.7、§6.2 的三条红线，立项时就打，别等出事）：

```
cd $TLP_HOME
sed -i 's|"DEPN\\t"kit_depend|"DEPN\\tKIT=\\t"kit_depend|" csh/tlp_pack.awk # ① 让依赖生效
# ②（可选）让退出码可用 ③ 统一 gawk —— 见《使用手册》附录 C.4
```

为什么必须在 P0：① 之后所有由 tlp_pack 生成的包才带真实依赖；② 决定 CI 能不能自动化。晚打补丁 = 之前发布的一批包永远没有依赖保护。

12.2 P1 首批收货：Foundry 的 6 个套件变成发布物

管理者按 §10.2 走完，落到命令上是这四步（全部实测）：

```
cd $ROOT # 演示根目录

# ① 裸目录 → 清单+包
tlp_pack --packCfgsDir release/configs --packDestDir release/packages \
--tempDir release/tempLib release/lists/T28HPC_r061.csv
# [tlp_pack]: Total 6/6 tlp packages are created.

# ② 登记 catalog
tlp_import --packageCfgDir release/configs --packageSrcDir release/packages --dataSheetDir release/dataSheets
# [tlp_import]: Total 6/6 tlp data sheets are created.

# ③ 参考库（管理者自己的“金标准”）
tlp_install --packageCfgDir release/configs --packageSrcDir release/packages \
--targetLibDir designer/techLib --bundleList T28HPC_r061.bundle
# [tlp_install]: Total 6/6 kits are installed.
```

```
# ④ 自检 + 冻结成 golden
bash tlp_verify.sh designer/techLib release/packages # RESULT: 全部通过
cp designer/techLib/T28HPC/Op5/.tlp_packages release/golden_r061.bundle
```

P1 结束时的资产清单（这就是「TLM 管理档案资料」的实体）：

```
release/lists/T28HPC_r061.csv    6 行、11 列，人的意图
release/configs/*.tlp (6)       包定义（机器可解析）= 发布记录本体
release/packages/*.tgz (6)      包实体（只读，SKU 不可变）
release/dataSheets/...*.dts (6) catalog：设计师能选到什么
release/dataSheets/.tlp_package_info.csv catalog 索引
T28HPC_r061.bundle             集合清单（安装顺序）
designer/techLib/               金标准参考库（含 .tlp_install/.tlp_packages/summary）
```

12.3 P2 环境铺开：9 名工程师同时上车

管理者发通告（§10.7）+ 给 `/proj/Alpha/meta/techLib.bundle`（= 金标准 bundle）。每个人执行 §11.1 的三条命令。要点：

- 全员用同一份 bundle，因此 techLib 内容可预期一致；差异只会来自 SDIR、挂载与时序。
- 每人跑 `tlp_verify.sh` 并留一份 stdout 在项目 wiki（这是最便宜的“环境一致性证据”）。
- 有人只要最小集（§11.2）→ 允许，但项目基线 bundle 不变（他额外记录自己的 subset bundle）。

一次“全员重装”演练（管理者主动做，成本极低、收益极大）：

```
foreach u (zhang li wang ...)
  tlp_install --packageCfgDir $REL/configs --packageSrcDir $REL/packages \
    --targetLibDir /proj/Alpha/$u/techLib --bundleList $BASE >/dev/null
  bash tlp_verify.sh /proj/Alpha/$u/techLib $REL/packages | tail -1
end
```

因为幂等，第二次跑的完整输出就是 `Total 6/6 kits are skipped. (already exist)`——“跳过”正是我们想要的证明。

12.4 P3 hotfix 阶段：三个补丁包的真实管理动作

Foundry 连发三次 CTK 修正（只给差异目录）。管理者：

```
# 清单：SDIR 沿用 ctk28_r061；REQU 指向“上一个 HF”（链式）
$EDITOR release/lists/CTK_hf.csv # 2 行 PATCH
tlp_pack --packCfgsDir release/configs --packDestDir release/packages --tempDir release/tempLib release/lists/CTK_hf.csv
# [tlp_pack]: Total 2/2 tlp packages are created.
tlp_import --packageCfgDir release/configs --packageSrcDir release/packages --dataSheetDir release/dataSheets
# [tlp_import]: Total 2/8 tlp data sheets are created. ← 新增 HF1/HF2
# [tlp_import]: Total 6/8 tlp files are skipped. (exist) ← 其余 6 个包早已登记
```

工程师追升级（追加而非重排）：

```
echo PT28HPCKITCTK_r0.6.1HF1 >> /proj/Alpha/meta/techLib.bundle # 依序 HF1→HF2
tlp_install --targetLibDir techLib --bundleList /proj/Alpha/meta/techLib.bundle ...
# : Directory '.../ctk28_r061' already exist.
# INFO: Unpacking file '...PT28HPCKITCTK_r0.6.1HF1.tgz' ...
# [tlp_install]: Total 2/2 kits are installed.
```

三个“教科书式”的护栏在此阶段全部被实测触发过：

- 1 跳装被拦：Required kit 'PT28HPCKITCTK_r0.6.1HF1' has not been installed yet. → (dependency fail)
- 2 基线缺失被警告：WARNING: Base directory 'ctk28_r061' is missing for patch package.
- 3 拿 FULL 重发同一 SDIR 会炸（管理者若想“重打包一份 r0.6.2 覆盖”）：
ERROR: Kit directory 'ctk28_r061' already exist before installing full kit package. + Total 1/1 kits have conflict root. (error)
→ 唯一正解是 PATCH，或改用新 SDIR + 新 SKU 并与设计师一起评估 EDA 挂载改动 (§11.7)。

工程师升级后的自查（不靠信任靠差异）：

```
$ find techLib -newermt '-1 day' -type f ! -name '*.dts' | sort
techLib/T28HPC/Op5/FDK/CTK/ctk28_r061/FIX.hf1
techLib/T28HPC/Op5/FDK/CTK/ctk28_r061/FIX.hf2
```

12.5 P4 Tapeout 冻结：四件套 + 只读 + 校验和（本案例的心脏）

Tapout 前 3

天，管理者与项目负责人一起做唯一一次“资料定版”。冻结动作请写成脚本执行——循环、子 shell、相对校验和这三件事在 csh 里都很别扭，所以本例用 bash (TLM 命令本身照旧由 shebang 走 csh，两语言混用没问题)：

```
A=archive/projAlpha_T28HPCOp5_TAPEOUT
mkdir -p $A/configs $A/packages

# ① bundle：从“验收机”的 techLib 抽，只允许【保序去重】，绝对不可 sort
awk '!seen[$0]++' /proj/Alpha/gold/techLib/T28HPC/Op5/.t1p_packages > $A/techLib.bundle

# ② 把清单点到的包与定义真正拷进档案（不是引用发布盘！）
cd $A
while read -r s; do
  cp "$ROOT/release/configs/$s.t1p" configs/
  cp "$ROOT/release/packages/$s.tgz" packages/
done < techLib.bundle

# ③ 校验和：在各自目录内相对生成 (§8.3——在外面跑会全线 FAILED)
(cd packages && md5sum *.tgz > ../packages.md5)
(cd configs && md5sum *.t1p > ../configs.md5)

# ④ 档案自检 + 上锁 + 归档指纹
(cd packages && md5sum -c ../packages.md5 | grep -c ': OK') # → 8
(cd configs && md5sum -c ../configs.md5 | grep -c ': OK') # → 8
find . -type f ! -name ARCHIVE.sha256 | sort | xargs sha256sum > ARCHIVE.sha256
chmod -R a-w configs packages
```

要在 csh 里做同样的事，就把 ② 的循环换成 `foreach s ( \awk '{print $1}' $A/techLib.bundle ) ... end` (csh 用反引号做命令替换，**没有 \$()**)，③ 的子 shell 要写成 `(cd ...)`` 转义。这正是我们建议“冻结流程脚本化 + 用 bash 写”的原因。

冻结档案最终长这样（20 个文件，实测）：

```
archive/projAlpha_T28HPCOp5_TAPEOUT/
├── techLib.bundle      ← 顺序即依赖序 (8 行)
├── packages.md5 configs.md5 ARCHIVE.sha256
├── configs/ (8 个 .t1p, 只读)
└── packages/ (8 个 .tgz, 只读)
```



```
$ cat -n techLib.bundle
 1PT28HPCKITPDK_r0.6.1
 2PT28HPCKITCTK_r0.6.1
 3PT28HPCKITADF_r0.6.1
 4SC6T_base_e.1.0
 5MEM2PRF_r.0.6.1
 6GPIO_e.0.6.1
 7PT28HPCKITCTK_r0.6.1HF1
 8PT28HPCKITCTK_r0.6.1HF2
```

为什么这个顺序不能动——同一批包，只差一个 `sort`：

bundle	结果
原序（安装序）	Total 8/8 kits are installed.
<code>sort</code> 之后	Total 2/8 kits are installed. + Total 6/8 kits require check fail. (error)

这就是 §6.7 的铁证：对 TLM 而言，清单顺序就是依赖图。

同时把源码级的东西纳入版本控制（`configs/` 已含全部信息，进 Git 才查得到“谁改了哪个包的定义”）：

```
git add archive/projAlpha_T28HPC0p5_TAPEOUT/{techLib.bundle,*.md5,configs/} && git commit -m "alpha: tapeout freeze r0."
```

冻结后任何改动 SKU 内容的行为都是事故。上锁后连 `rm` 都会被拒（实测 `Permission denied`）。真要改必须新 SKU + 新档案目录。

12.6 P5 复现证明：半年后凭档案重装（管理者 + 新人各跑一次）

管理者验收（新机器/新环境，不依赖发布盘）：

```
set A = $ROOT/archive/projAlpha_T28HPC0p5_TAPEOUT
tlp_install --packageCfgDir $A/configs --packageSrcDir $A/packages \
  --targetLibDir $ROOT/repro/techLib.repro --bundleList $A/techLib.bundle
# [tlp_install]: Total 8/8 kits are installed.
bash tlp_verify.sh $ROOT/repro/techLib.repro $A/packages
# ---- ../techLib.repro/.tlp_install 共 8 条记录
# RESULT: 全部通过
```

新人只拿冻结树（完全不碰 `release/`）：

```
cp $A/techLib.bundle .
tlp_install --packageCfgDir $A/configs --packageSrcDir $A/packages --targetLibDir techLib --bundleList techLib.bundle
# [tlp_install]: Total 8/8 kits are installed.
bash tlp_verify.sh techLib $A/packages # ---- techLib/.tlp_install 共 8 条记录 / RESULT: 全部通过
diff -r -x '.tlp_install*' -x '.tlp_packages' $ROOT/designer/techLib techLib
# 无差异 → 冻结 = 可完整复现
```

`diff` 必须带那两条 `-x`（否则 `.tlp_install.summary` 里的时间戳/用户名/源路径必然不同，那是噪声不是差异）。

12.7 P6 回片 ECO：冻结之后又来了一个 hotfix

原则：`*_TAPEOUT` 档案永不改（§10.5）。ECO 用新档案目录表达增量：

```
# 管理者：照常走发布链，产出 HF3 (§10.3)
tlp_pack ... release/lists/CTK_hf3.csv          # Total 1/1 tlp packages are created.
tlp_import ...                                # Total 1/9 created / 8 skipped
# 建 ECO 档案 = 冻结 bundle + 追加 HF3 (顺序：老 8 行后加第 9 行)
set E = archive/projAlpha_T28HPCOp5_ECO1
cp -r $A $E && chmod -R u+w $E
echo PT28HPCKITCTK_r0.6.1HF3 >> $E/techLib.bundle
cp release/configs/PT28HPCKITCTK_r0.6.1HF3.tlp $E/configs/
cp release/packages/PT28HPCKITCTK_r0.6.1HF3.tgz $E/packages/
(cd $E/packages && md5sum *.tgz > ../packages.md5)
(cd $E/configs && md5sum *.tlp > ../configs.md5)
```

设计师在 ECO 工作区重装（幂等 + 依赖链正确，实测）：

```
      : Directory '.../ctk28_r061' already exist.
INFO: Unpacking file 'release/packages/PT28HPCKITCTK_r0.6.1HF3.tgz' ...
[tlp_install]: Total 1/1 kits are installed.
---- designer/techLib/.tlp_install 共 9 条记录
RESULT: 全部通过
```

审计口径因此非常干净：TAPEOUT = 8 个包（流片用的真相），ECO1 = 9 个包（现在改的真相），两份 bundle 都是自解释、可复现、带校验和的。谁也不需要“记得当时装了什么”。

12.8 P7 年度审计与退役（30 分钟做完）

```
# 档案盘点：每个档案几个包（csh 写法，`wc -l <` 只输出数字）
foreach d (archive/*/ )
  echo -n "$d "
  wc -l < $d/techLib.bundle
end
# 档案完整性抽查（sha 清单）
cd archive/projAlpha_T28HPCOp5_TAPEOUT && sha256sum -c ARCHIVE.sha256 | grep -v ': OK'
# 日志/暂存清理（日志会无限轮转，别让它塞满盘）
rm -f $ROOT/*.log.[0-9]* ; rm -rf $ROOT/release/tempLib
# EOL：见 §10.8（只在 .tlp 的 END 之后加 DEPRECATED 说明，绝不删包）
```

审计表（本项目一年之末的实际状态，可直接当模板）：

档案	包数	冻结时点	md5 全 OK	复现验证
projAlpha_T28HPCOp5_TAPEOUT	8	W26	8/8 + 8/8	diff -r 零差异
projAlpha_T28HPCOp5_ECO1	9	W38	重算	verify 通过（9 条）

12.9 案例复盘：五条被验证过的经验

- 清单顺序 > 一切智能。同一批 8 个包，8/8 与 2/8+6/8 error 的唯一差别是有没有保持 .tlp_packages 的原序。
- “跳过”是最好的日志。幂等（kits are skipped. (already exist)）让“重跑一遍确认”成为零风险验收手段——每个阶段我们都重跑了一次。
- import 是最好的门禁。缺载荷就不进 catalog（Total 0/1 created + missing package files. (error)），把质量问题挡在发布环节。

- 4 冻结必须是“四件套 + 只读 + 校验和”。只留 bundle 不够（包会被删）；只留包不够（定义会漂移）。本案例 20 个文件的档案是复现成功的充分条件。
- 5 工具的沉默就是它的谎言。退出码恒 0、-v 无效、MULTI 死代码、DEPN KIT=<SKU> 永久失败（实测矩阵在《使用手册》§5.3）、tlp_check 静默——所以 P0 必须打 §4.7 补丁 + 把 verify/gate/topo 纳入流程，否则前 6 个月你会一直在“看起来装成功了”的状态下工作。

附录

附录 A 管理者一页速查卡

```
setenv TLP_HOME /tools/tlm ; set path = ($TLP_HOME/bin $path) # 每次登录
setenv REL /tlppkg/release/T28HPC/Op5 # 当前发布根

# 发一次包
tlp_pack --packCfgsDir $REL/configs --packDestDir $REL/packages --tempDir $REL/tempLib $REL/lists/r071.csv
mv T28HPC_r071.bundle $REL/bundles/ # ★bundle 落在 CWD
bash gate.sh # .tlp/.tgz 静态检查
tlp_import --log imp.log --packageCfgDir $REL/configs --packageSrcDir $REL/packages --dataSheetDir $REL/dataSheets
grep -q '(error)' imp.log && echo FAIL # ★退出码没用
tlp_install --packageCfgDir $REL/configs --packageSrcDir $REL/packages --targetLibDir /tmp/ref --bundleList $REL/bundles/x
bash tlp_verify.sh /tmp/ref $REL/packages # 装后自检
chmod -R a-w $REL/packages $REL/configs # 发布即不可变

# hotfix：清单第一格写 PATCH，SDIR=基线，REQU=上一个 HF，然后同上四步
# 冻结：awk '!seen[$0]++' 抽 bundle → 拷 configs/packages → md5x2 → sha256 → chmod a-w
# 审计：ls $REL/dataSheets 结构 / find -name '<SKU>.dts' / grep -l '^DEPN.*<SKU>' $REL/configs/*.tlp
```

附录 B 设计工程师一页速查卡

```
setenv TLP_HOME /tools/tlm ; set path = ($TLP_HOME/bin $path)
set P = /tlppkg/release/T28HPC/Op5

tlp_install --packageCfgDir $P/configs --packageSrcDir $P/packages --targetLibDir ~/techLib --bundleList /proj/Alpha/meta/te
bash /tools/tlm/contrib/tlp_verify.sh ~/techLib $P/packages # ← 开工前

# 查：ls ~/techLib/.tlp_install/ (权威已装集)
# cat ~/techLib/T28HPC/Op5/.tlp_packages (含顺序，可当 bundle)
# tail ~/techLib/.tlp_install.summary (谁在何时装了什么)
# more .../<SDIR>/<SKU>.dts (这个库的定义/版本)
# 升级：echo <新SKU> >> /proj/Alpha/meta/techLib.bundle 后重跑上面的 install (幂等)
# 卸载：见 §8.5 (三处一起删 + verify)
# 禁忌：不要 sort bundle · 不要手删 .dts/.tlp_install · 不要在路径里放空格
# · 不要 `cp -r` 别人的 techLib · 不要用交互菜单建正式库
```

附录 C 场景 → 命令矩阵

我要做的事	命令（含必带选项）	见
把交付目录变成包	tlp_pack --packCfgsDir --packDestDir --tempDir <csv>	§4.4

我要做的事	命令（含必带选项）	见
只发 hotfix	<code>tlp_pack</code> (CSV 用 PATCH + SDIR=基线 + REQU=上个 HF)	§4.6
让新包能被选到	<code>tlp_import --packageCfgDir</code> <code>--packageSrcDir --dataSheetDir</code>	§5.3
只登记一个包	<code>tlp_import <...>/configs/<SKU>.tlp</code>	§5.2
发布 QA	<code>tlp_import</code> 后 <code>grep '(error)' + gate.sh</code>	§5.5/§8.2
一键建我的库	<code>tlp_install --bundleList </code> <code>--targetLibDir <t> --packageCfgDir</code> <code>--packageSrcDir</code>	§6.3
试装单个包	<code>tlp_install <SKU></code>	§6.4
图形化挑包	<code>tlp_install --dataSheetDir</code> <code><catalog></code> (不给包名)	§6.5
追 hotfix	<code>bundle</code> 末尾追加 + 重跑 <code>tlp_install --bundleList</code>	§6.6/§11.3
项目换基线	新 <code>--targetLibDir</code> + 新 <code>bundle</code> (别混装同一目录)	§11.5
让装单个库时自动补齐基线	在包 <code>.tlp</code> 里写 <code>REQU <基线SKU></code> <code><基线SKU></code> (同名两遍)	§6.12
看装了什么	<code>ls <t>/.tlp_install / cat</code> <code><t>/.../.tlp_packages</code>	§11.4
装后完整性检查	<code>bash tlp_verify.sh <t> <pkgs></code>	§8.1
卸一个库	手删三处 + <code>verify</code>	§8.5
生成可复现清单	<code>awk '!seen[\$0]++'</code> <code>.../.tlp_packages > x.bundle</code> (不可 sort)	§6.7
依赖自动排序	<code>bash topo.sh <cfs> <SKU...></code>	§8.4
Tapeout 冻结	<code>bundle + configs/ + packages/ + 2xmd5 + sha256 + chmod a-w</code>	§12.5
复现/交接	只读档案 + <code>tlp_install --bundleList</code> + <code>diff -r -x</code>	§12.6
装后校验 (工具自带)	—— 不存在 (<code>tlp_check</code> 静默无操作)	§7.1

版本与校验信息

项	值
代码基线	commit 8072710 , gitee.com/icdop/tlp == github.com/icdop/tlm (逐文件一致；你给的 gitee.com/icdop/tlm 需鉴权)
运行环境	Linux x86_64 · csh 20240808 · GNU Awk 5.2.1 · GNU tar · tree
本案例覆盖	tlp_pack (全量/hotfix/冲突重发)、tlp_import (全量/增量/门禁/幂等)、tlp_install (bundle/包名/交互/幂等/依赖失败/冲突/最小集/跨源)、tlp_check、tlp_help、冻结与复现、卸载与回退
尚未覆盖	Cygwin/老 tar 分支、MULTI 之外的 pack 罕见路径 (--test 模式已验证其“文件级开关”副作用)
与《使用手册》的分工	手册给“为什么”“有哪些坑”(32 条实测清单 + .tlp 完整语法矩阵)；本书给“按岗位怎么做”“每个命令怎么用”，案例复用同一套证据链

参考书完